You can then start a PostgreSQL container, specifying that it should be part of the `catalog-network` you just created. Using the `--net` argument ensures the container will join the specified network and rely on the Docker built-in DNS server:

```
$ docker run -d \
    --name polar-postgres \
    --net catalog-network \
    -e POSTGRES_USER=user \
    -e POSTGRES_PASSWORD=password \
    -e POSTGRES_DB=polardb_catalog \
    -p 5432:5432 \
    postgres:14.4
```

If the command fails, you might have the PostgreSQL container from chapter 5 still running. Remove it with `docker rm -fv polar-postgres` and run the previous command again.

6.2.2 Containerizing Spring Boot with Dockerfiles

Cloud native applications are self-contained. Spring Boot lets you package your applications as standalone JARs, including everything they need to run except the runtime environment. That makes the containerization very straightforward, since all you need in a container image besides the JAR artifact is an operating system and a JRE. This section will show you how to containerize the Catalog Service application using a Dockerfile.

First, you need to identify which image you want to base yours on. You could choose an Ubuntu image, as we did before, and then explicitly install the JRE, or you could choose a base image that provides a JRE already, which is more convenient. All the major OpenJDK distributions have a related image available on Docker Hub. Feel free to choose the one you prefer. In this example I'll be using Eclipse Temurin 17, which is the same